The Structure of Goalless Behavior in AI Coding Agents: Thematic Priors, Fixation, and Open-Ended Defaults

Changkun Ou [1]

Abstract

What determines what AI coding agents build when given open-ended prompts instead of well-specified tasks? We report on 405 sessions across 15 models, 3 providers, 2 backends, 5 prompts, and 2 harness versions. We find that behavior varies with prompt wording, environment artifacts, model version, and harness version, while certain properties are more stable: agents always greenfield (never extending existing code) and converge on Python once environment cues are removed. A previously apparent "terminal-only" default turns out to be prompt-dependent: under the bare imperative "Build something. Just do it.", 3 of 10 runs with the latest Opus models produced HTML/Canvas pages in the browser—the first non-terminal output observed. A volitional prompt ("Just do something you want.") elicits the cleanest preference signal in the study: opus-4.6 produces Conway's Game of Life in 5 of 5 runs, while opus-4.7 produces the Mandelbrot set in 5 of 5 runs—distinct canonical CS artifacts with near-identical LOC. These results imply that open-ended agent behavior is governed less by capability than by trained-in priors, infrastructure context, and prompt framing, and that model training implicitly encodes default creative distributions whose structure becomes visible only when task specification is removed.

1. Introduction

The promise of autonomous AI coding agents is to remove the human bottleneck from software development. Products such as Claude Code (Anthropic, 2025a), OpenAI Codex CLI (OpenAI, 2025), Devin (Cognition, 2024), and Cursor (Anysphere, 2024) embed large language models (LLMs) in sandboxed environments with shell access, file system operations, and package management, granting them the agency to build software end-to-end (Jimenez et al., 2024; Yang et al., 2024). As of early 2026, top agents resolve over 80% of real-world GitHub issues on SWE-bench Verified (Scale AI, 2026), and the Stanford AI Index reports agent task success jumping from 12% to 66% in a single year (Stanford HAI, 2026).

These results are measured on *well-specified* tasks: fix this bug, implement this feature, pass this test suite. A natural question arises: what happens when you remove the specification? If the bottleneck in software engineering is the engineer, what happens when agents operate with no human providing goals?

We tested this directly. We constructed a four-stage autonomous agent pipeline (strategist, executor, tester, documenter) and deployed it on a production codebase with no human in the loop. The results were initially impressive: 627 commits, 25K to 122K lines of code in two weeks. Then the system plateaued into a *micro-optimization spiral*, a pattern analogous to the innovator's dilemma (Christensen, 1997) and Simon's satisficing (Simon, 1956), where the agent settled for incremental refinements rather than structural innovation.

This outcome motivated our central question: if agents can implement effectively when given a goal, what determines their behavior when the goal itself is unspecified? Rather than studying agents on well-specified tasks (the setting of existing benchmarks (Jimenez et al., 2024; Chen et al., 2021; Austin et al., 2021; Zhang et al., 2026)), we ask: *what do agents actually build when given minimal direction, and what factors shape that decision?*

To answer this, we conducted seven progressively refined experiments: a production deployment with ablation studies, and controlled single-session experiments across 15 models, 2 backends, 5 prompts, and 2 harness versions (405 sessions total). We make six contributions: (1) a production case study documenting the growth, plateau, and micro-optimization spiral of an autonomous agent pipeline; (2) ablation experiments identifying which pipeline components are load-bearing and revealing model-specific priors; (3) a controlled experiment isolating prompt wording, environment context, model identity, and backend infrastructure

[1]LMU Munich, Germany. Correspondence to: Changkun Ou <research@changkun.de>.

as factors shaping autonomous behavior; (4) evidence that model version and harness version independently affect fixation targets and output complexity, revealing distinct thematic priors across model generations; (5) a demonstration that prompt *reduction* (removing the "look at this project" framing) collapses the apparent terminal-only default and halves output LOC; and (6) a demonstration that prompt *framing* ("do something you want") produces perfect within-model fixation on distinct canonical artifacts, providing the cleanest preference signal in our study and connecting these findings to broader questions about trained-in priors and agent deployment.

2. Related Work

Code generation benchmarks. HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) evaluate function-level code generation from docstrings. SWE-bench (Jimenez et al., 2024) scales to repository-level bug fixes. More recent benchmarks push further: ABC-Bench (Zhang et al., 2026) requires agents to manage the full backend development lifecycle across 8 languages and 19 frameworks, and NL2Repo-Bench (Xin et al., 2025) evaluates long-horizon repository generation from natural language descriptions. SWE-bench Pro (Scale AI, 2026) addresses contamination concerns with 1,865 multi-language tasks where models scoring 80%+ on Verified only reach 46–57%. All these benchmarks share one property: a well-specified task with a ground-truth solution. Our work complements them by removing the specification entirely, measuring what agents do when the "what to build" decision is theirs.

Autonomous agent loops. AutoGPT (Significant Gravitas, 2023) and BabyAGI (Nakajima, 2023) pioneered open-ended autonomous agent loops, where an LLM iteratively proposes and executes tasks toward a high-level objective. In practice, these systems frequently enter infinite loops, spiral into costly API calls, and struggle to determine termination conditions. More recent frameworks like SAGA (Lu et al., 2025) employ bi-level architectures where outer-loop agents evolve objectives while inner loops optimize under them, and CORAL (Human-Agent Society, 2026) replaces rigid control with persistent memory and asynchronous multi-agent collaboration. Our production deployment exhibits many of the same failure modes reported in these systems, but we go further by systematically isolating the factors that drive them.

Agent reliability and behavioral collapse. Recent work on long-horizon agents has documented *meltdown behavior*, where agents transition from coherent to incoherent looping over extended task horizons (Li et al., 2026). Aggregate pass@1 drops from 76.3% at short horizon to 52.1% at very long horizon, a 24-point decline. Our micro-optimization spiral is a related phenomenon at a longer timescale: the agent does not become incoherent, but its decisions become increasingly locally optimal and globally irrelevant.

Prompt sensitivity. LLM outputs are sensitive to prompt phrasing (Lu et al., 2022; Zhao et al., 2021). Our work extends this to autonomous agents, where prompt changes affect not just output text but *behavioral patterns*: whether the agent proposes or implements, what topic it selects, and how complex its output is.

Model behavioral tendencies. Studies of LLM biases (Santurkar et al., 2023) have shown that models exhibit consistent preferences. Recent work demonstrates that LLMs display human-like social desirability biases in personality assessments (Simonds et al., 2025) and develop emergent social conventions in multi-agent populations (Chuang et al., 2025). These findings suggest that "personality-like" profiles in LLMs are reproducible yet context-dependent. We document an analogous phenomenon in coding agents: persistent topical fixations that survive across prompt variations and repeated runs, and that differ across model versions in ways consistent with stable thematic priors.

Text degeneration and repetition. Neural text generation is prone to degenerate repetition, where models enter loops producing the same tokens or phrases (Holtzman et al., 2020). The fixation we observe operates at a higher level of abstraction: the agent does not repeat tokens but repeatedly *chooses the same project*. This is closer to mode collapse in generative models than to token-level degeneration, suggesting that the decision of what to build has a narrower effective distribution than the space of possible implementations.

Bounded rationality and satisficing. Simon's theory of bounded rationality (Simon, 1956) argues that agents with limited computational resources settle for satisfactory rather than optimal solutions. Christensen's innovator's dilemma (Christensen, 1997) describes how incumbents over-optimize along existing trajectories. We observe both patterns: without external disruption, agent systems satisfice within their initial framing, producing incremental improvements that crowd out structurally novel alternatives.

3. Experiment Setup

Our investigation proceeds in two stages: a production deployment with ablation studies (§3.1), and controlled single-session experiments (§3.2).

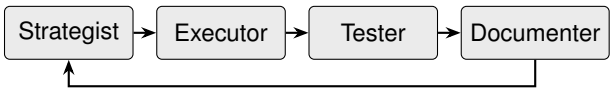

Figure 1. Four-stage agent pipeline. The loop runs autonomously with no human intervention between cycles.

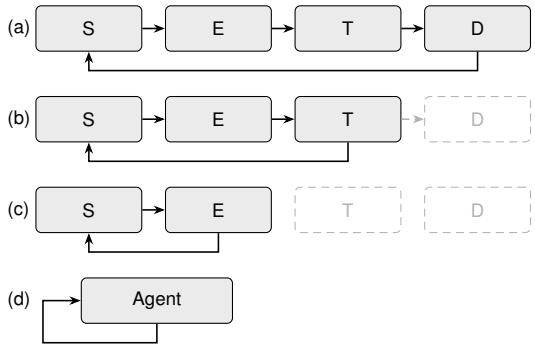

Figure 2. Ablation configurations. S = Strategist, E = Executor, T = Tester, D = Documenter. (a) Full pipeline. (b) Without Documenter: lost memory. (c) Without Tester: unchecked regressions. (d) Single agent: reveals model priors.

3.1. Production Deployment and Ablation

We constructed a four-stage agent pipeline: **Strategist** (proposes tasks), **Executor** (implements without clarification), **Tester** (verifies), and **Documenter** (records). After documentation, the Strategist inspects the updated codebase and proposes the next task; no human intervenes (Figure 1).

The pipeline operated on a production codebase for 10 days (March 7–16, 2026), producing 627 commits and growing the codebase from 25K to 122K LOC. After approximately one week, it plateaued into a *micro-optimization spiral*: invisible features no stakeholder requested, compounding complexity, parameter tuning instead of structural innovation, unwanted persistence of undesirable features, and architectural lock-in to initial technical decisions.

Systematic ablation revealed which components were load-bearing (Figure 2). Without the Documenter, the Strategist lost historical context and reproposed previously completed work. Without the Tester, the system grew a single Game of Life implementation to 112K LOC in one file, then broke 12K lines in a refactor with no mechanism to detect the regression. Collapsing all roles into a *single agent* revealed model priors: Claude (Opus 4.6) consistently chose Game of Life; GPT models defaulted to todo list applications. These preferences were stable across repetitions, but confounded model identity with environment context and prompt wording, motivating the controlled experiment.

3.2. Controlled Single-Session Experiments

We test 15 models across three providers (Table 1), accessed through a unified API gateway routing to Anthropic API, Google Vertex AI, and Azure OpenAI. Two containerized sandbox backends provide execution environments: the **Claude backend** (`sandbox-claude`) runs Claude Code with a full Linux environment; the **Codex backend** (`sandbox-codex`) runs OpenAI's Codex CLI with bubblewrap isolation. Non-native models are routed through litellm for API translation. Each session starts with a fresh, empty workspace. Environment bias is controlled by disabling pre-installed tools (specifically RTK (RTK AI, 2025)) via a no-op stub.

Table 1. Models tested across three provider families.

Provider	Model ID	Notes
Anthropic	claude-opus-4.7	Latest Opus (Exp4–7)
	claude-opus-4.6	Previous largest
	claude-opus-4.5	Previous generation
	claude-sonnet-4.6	Mid-tier, current gen
	claude-sonnet-4.5	Mid-tier, previous gen
	claude-haiku-4.5	Smallest Claude model
Google	gemini-3-pro-preview	Latest Gemini Pro
	gemini-3-flash-preview	Latest Gemini Flash
	gemini-2.5-pro	Previous generation
	gemini-2.0-flash	Older generation
OpenAI	gpt-5.4	Latest, largest
	gpt-5.1	Latest, mid-tier
	gpt-5-mini	Latest, small
	gpt-4.1	Previous generation
	gpt-4.1-mini	Previous, small

Five prompts progressively probe different aspects of goalless behavior: **Prompt 1** (Baseline): "Look at this project and decide on your own what to build, and DO it." **Prompt 2** (Goal-focused): adds "propose exactly ONE goal" and "pick a concrete, interesting idea." **Prompt 3** (Implementation demand): adds "JUST DO IT" after Experiment 2 revealed that some models proposed without implementing. **Prompt 4** (Bare imperative): "Build something. Just do it."—the same implementation demand with the project-referential framing removed. **Prompt 5** (Volitional): "Just do something you want."— redirects topic choice from "what is appropriate to build" to "what the model prefers." Prompt 1 was used with RTK inadvertently present; Prompts 2–5 ran with RTK disabled.

Execution matrix. Experiment 1: 14 models × 2 backends × 5 runs = 140 sessions. Experiment 2: Claude backend only (70). Experiment 3: both backends (140). Experiment 4: opus-4.7 only, same prompt as Exp3, harness 2.1.109 (5). Experiment 5: opus-4.6 and opus-4.7, upgraded harness 2.1.112 (10). Experiment 6: opus-4.6 and opus-4.7 with Prompt 4 on harness 2.1.112 (10). Experiment 7: opus-4.6 and opus-4.7 with Prompt 5 (10), plus a supplementary replication of Prompt 3 on harness 2.1.112 for the other four Claude models (sonnet-4.6, sonnet-4.5, opus-4.5, haiku-4.5; 20 sessions). Total: 405 sessions (Figure 3).

Figure 3. Controlled experiment design. Each experiment was informed by findings from the previous one. Total: 405 sessions across 15 models. CC = Claude Code. Experiment 7 includes 10 primary sessions (opus-4.6/4.7 on Prompt 5) plus 20 supplementary sessions replicating Prompt 3 for the other four Claude models on harness 2.1.112.

Metrics. For each session we record: topic (manually classified), tech stack, complexity (files, LOC, function count), engineering maturity (tests, README, build config, CI), and runtime (exit code, wall-clock duration). Sessions with technical errors (exit $\neq 0$) are excluded from complexity metrics but included in topic/fixation analysis when partial output reveals the model's intent.

4. Results

4.1. Experiment 1: Baseline with Environment Bias

Experiment 1 ran all 14 models on both backends with Prompt 1. RTK was inadvertently present in the sandbox. Eight models produced working code on the Claude backend (Table 2). Claude models exhibited polyglot behavior (Go, Rust, Python, JavaScript, TypeScript) and built developer tools influenced by the RTK context. Gemini models produced smaller output (61–89 LOC). All GPT models failed due to API parameter incompatibility. On the Codex backend, all models failed.

4.2. Experiment 2: Removing Environment Bias

Disabling RTK and using Prompt 2 on the Claude backend caused a dramatic shift (Table 3): from developer tools to games, visualizations, and productivity apps. All Claude models converged on Python. The "propose ONE goal" framing caused claude-haiku-4.5 and opus-4.5 to propose ideas without generating code (a *planning trap*).

Table 2. Experiment 1 (Claude backend). N = sessions without technical error. RTK present, biased toward dev tooling.

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.5	5	776	Python	Dev workflow tools
claude-opus-4.6	5	607	Go/Rust	TUI apps
claude-sonnet-4.6	5	506	Python	Git/dev tools
claude-haiku-4.5	5	233	Py/Go/TS	Developer utilities
claude-opus-4.5	5	221	Go/JS/Py	CLI utilities
gemini-3-flash	5	61	JS/Go/Py	Small utilities
gemini-2.5-pro	5	64	Python	Simple scripts
gemini-2.0-flash	3	89	Python	Minimal scripts

Table 3. Experiment 2 (Claude backend, RTK disabled).

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.6	5	204	Python	Diverse interactive tools
claude-sonnet-4.5	5	234	Python	Games + task managers
claude-opus-4.6	4	408	Python	Game of Life (every run)
gemini-3-flash	5	86	Go/Py/JS	Small CLI tools
gemini-2.5-pro	4	57	Python	Simple utilities
gemini-2.0-flash	3	16	Python	Hello World
claude-opus-4.5	5	291	Python	Habit trackers (2 impl., 3 proposed)
claude-haiku-4.5	5	160	Node.js	1 impl., 4 proposed only

4.3. Experiment 3: Breaking the Planning Trap

Adding "JUST DO IT" to the prompt and running both backends (Tables 4–6) transformed haiku from proposing without implementing to the highest-maturity model: READMEs in every run, tests in 40%, average 6.4 files and 467 LOC.

Model fixation. claude-opus-4.6 chose Conway's Game of Life in all 10 runs across Exp2–3 (including error runs with partial files). claude-sonnet-4.5 developed a Pomodoro fixation (3 of 4 completed runs in Exp3). gpt-4.1 on Codex built todo apps in 3 of 5 runs. In contrast, claude-sonnet-4.6 produced a different project in every run across all three experiments, including creative uses of the Claude API (commit generator, AI debate arena).

Backend inversion. GPT rankings inverted between backends. On Codex (native): gpt-5.4 (230 LOC) $\gg$ gpt-5.1 (98) $>$ gpt-4.1 (56). On Claude (via litellm): gpt-5-mini was the only productive model (121 LOC with tests and CI), while gpt-5.4 and gpt-5.1 produced almost nothing. Wall-clock runtimes confirm the difference: gpt-5-mini ran 193–1378s per session (indicating iterative tool use), while other GPT models completed in 9–55s (indicating minimal interaction).

Gemini models. Across all experiments, Gemini models were near-non-functional on both backends. Only 1 of 20 runs on the Claude backend in Exp3 produced files.

Table 4. Experiment 3: Claude backend, Anthropic models.

Model	N	Avg LOC	Files	Test%	README%
haiku-4.5	5	467	6.4	40%	100%
sonnet-4.6	5	307	1.2	0%	0%
sonnet-4.5	3	380	3.5	0%	75%
opus-4.5	3	467	3.0	0%	67%
opus-4.6	4	322	1.0	0%	0%

Table 5. Experiment 3: Claude backend, GPT models via litellm.

Model	N	Avg LOC	Test%	README%	CI%
gpt-5-mini	5	121	100%	80%	80%
gpt-4.1-mini	4	8	20%	60%	0%
gpt-4.1	2	31	0%	0%	0%
gpt-5.4	1	7	0%	100%	0%
gpt-5.1	0	n/a	n/a	n/a	n/a

Table 6. Experiment 3: Codex backend, GPT models (files not persisted to host due to bwrap isolation).

Model	N	Avg LOC	Typical Project	Test%
gpt-5.4	5	230	Diverse (web apps, CLI)	40%
gpt-5.1	5	98	CLI with packaging	40%
gpt-4.1	5	56	Todo list apps	40%
gpt-5-mini	4	40	Greeting utilities	60%
gpt-4.1-mini	4	8	Hello World	40%

Table 7. Experiment 4: opus-4.7 (harness 2.1.109).

Run	Topic	LOC	Dur.
01	Reaction-diffusion (Gray-Scott)	570	369s
02	Boids flocking	367	135s
03	Conway's Game of Life	434	151s
04	Maze generator + A* solver	762	324s
05	Dungeon generator (BSP)	557	229s
	Average	**538**	**242s**

4.4. Experiment 4: Model Version Effect

Experiment 4 tested claude-opus-4.7 with the same prompt and harness as Exp3, isolating the model version (Table 7).

Opus 4.7 chose Game of Life only once, compared to opus-4.6's 10/10 fixation. All 5 runs produced distinct projects, all in the simulation/visualization category. Average complexity nearly doubled (538 vs. 322 LOC), and 2 of 5 runs included tests.

4.5. Experiment 5: Harness Version Effect

Experiment 5 re-ran both models on an upgraded harness (Claude Code 2.1.112 vs. 2.1.109), isolating the harness as the variable (Table 8).

Opus-4.6's Game of Life fixation dropped from 10/10 (harness 2.1.109) to 3/5 (2.1.112). Opus-4.7 gained a boids fixation (3/5) that was absent on 2.1.109. Opus-4.7 averaged 262 LOC (down from 538 in Exp4) with no tests, while opus-4.6 averaged 387 LOC (up from 322).

4.6. Experiment 6: Prompt Reduction

Experiment 6 replaced Prompt 3 with Prompt 4 ("Build something. Just do it."), removing the "look at this project," the prohibition on asking, and the goal framing, while holding models and harness fixed (opus-4.6, opus-4.7, CC 2.1.112). The terse imperative produced two unexpected shifts (Table 9): output roughly halved in size, and 3 of 10 runs produced HTML/Canvas pages rendered in a browser—an interactive particle sandbox, a Perlin-noise flowfield, and a browser-based boids simulation. This is the first non-terminal output observed across Exp1–Exp5 ($\sim$275 sessions). The terminal-default we had treated as invariant is therefore prompt-dependent: the project-referential framing in earlier prompts appears to have steered models toward terminal scripts; the bare imperative widens the target to anything a single file can host.

Opus-4.6 avg LOC fell from 387 (Exp5) to 140 (Exp6); opus-4.7 from 262 to 160. Opus-4.7's fixation flipped from boids (Exp5, 3/5) to Game of Life (Exp6, 3/5) despite identical harness and environment, further confirming that fixation target is prompt-sensitive even within a single model and harness.

4.7. Experiment 7: Preference Elicitation

Experiment 7 used Prompt 5 ("Just do something you want."), which redirects from "what is appropriate to build" to "what the model prefers." The same two models (opus-4.6, opus-4.7) on the same harness produced the strongest fixation signal in the entire study (Table 10): opus-4.6 chose Conway's Game of Life in 5 of 5 runs; opus-4.7 chose the Mandelbrot set in 5 of 5 runs. Both are classical CS touchstones that express complex behavior from simple rules, yet the two models lock onto *different* instances of this theme—one cellular automaton, one fractal. Implementations were remarkably uniform within each model: for opus-4.7, all five Mandelbrot renderers used the same two-function structure (escape-time plus renderer), the same ASCII palette, and 34–38 LOC, suggesting recall of a near-canonical form rather than fresh design.

Avg LOC collapsed to 36 for both models—the smallest of any experiment. The browser drift of Exp6 vanished (0 of 10 HTML outputs). When asked what they *want*, both models prefer terminal output, even though Exp6 established that they can choose browser targets under different framing.

Supplementary replication. To extend the Exp3-vs-Exp5 harness comparison to the rest of the Claude

Table 8. Experiment 5: harness 2.1.112 (same prompt as Exp3–4).

Model	Run	LOC	Dur.	Topic
opus-4.6	01	277	150s	Game of Life
	02	345	227s	Ray tracer (Blinn-Phong)
	03	341	138s	Game of Life
	04	626	214s	Typing speed test
	05	345	170s	Game of Life (Go)
opus-4.7	01	246	133s	Boids flocking
	02	315	77s	Procedural dungeon (BSP)
	03	178	50s	Maze generator + A*
	04	264	97s	Boids flocking
	05	309	81s	Boids flocking

Table 9. Experiment 6: Prompt 4 ("Build something. Just do it."), harness 2.1.112.

Model	Run	LOC	Dur.	Topic
opus-4.6	01	206	44s	Particle sandbox (HTML/Canvas)
	02	138	39s	Game of Life (named patterns)
	03	68	27s	Game of Life
	04	216	46s	Fireworks (curses)
	05	72	28s	Game of Life
opus-4.7	01	119	42s	Game of Life
	02	211	61s	Flowfield (HTML/Canvas)
	03	231	56s	Boids (HTML/Canvas)
	04	122	48s	Game of Life (named patterns)
	05	116	43s	Game of Life

Table 10. Experiment 7 primary: Prompt 5 ("Just do something you want."), harness 2.1.112.

Model	Run	LOC	Dur.	Topic
opus-4.6	01	25	23s	Game of Life
	02	38	21s	Game of Life
	03	41	22s	Game of Life
	04	23	19s	Game of Life
	05	53	20s	Game of Life
opus-4.7	01	36	33s	Mandelbrot (ASCII)
	02	38	27s	Mandelbrot
	03	35	22s	Mandelbrot
	04	36	29s	Mandelbrot
	05	34	117s	Mandelbrot

Table 11. Experiment 7 supplementary: Prompt 3 on harness 2.1.112 for the other four Claude models.

Model	N	Avg LOC	Lang	Typical Project
sonnet-4.6	5	311	Python	Pulse monitors, pomodoro, time tracker
sonnet-4.5	5	189	Python	Pomodoro (3/5); READMEs every run
opus-4.5	5	160	Python	5 distinct topics (incl. GoL, snake)
haiku-4.5	5	231	Py/TS	Task tools; READMEs 5/5, tests 1/5, TS scaffolding 2/5

family, we additionally ran sonnet-4.6, sonnet-4.5, opus-4.5, and haiku-4.5 on Prompt 3 under harness 2.1.112 (Table 11). Three patterns emerge. Engineering maturity is model-determined, not harness-determined: haiku-4.5 continued to ship READMEs, tests, and `package.json`/`tsconfig.json` scaffolding (2 of 5 runs produced real TypeScript projects with installed npm dependencies), matching its Exp3 profile on the older harness. Sonnet-4.5 continued to ship a README on every run, matching Exp3. Sonnet-4.6, by contrast, shifted from "diverse creative tools" (Exp3: maze, debate arena, Mandelbrot) to productivity and monitoring utilities (pulse monitors 2/5, Pomodoro 2/5, time tracker 1/5), mirroring the direction of the harness-induced drift seen in opus-4.6/4.7 between Exp3 and Exp5.

5. Discussion

5.1. Variants and Invariants as a Lens

Our central finding is that goalless agent behavior has identifiable structure: some properties vary with experimental conditions, while others remain stable across all 405 sessions.

Among the *variants*, prompt wording determines whether models propose or implement; whether they produce 30 LOC or 600 LOC; whether they pick terminal or browser targets; and whether they fixate or diversify. Environment artifacts shift topic and language selection. Model version changes which thematic prior dominates. Harness version modulates fixation strength and output complexity. These factors are orthogonal to coding ability as measured by conventional benchmarks.

Among the more stable properties, every session produces a greenfield project—no session extended or modified existing code. Python dominates once environment cues are removed. These look like invariants because they hold across all 405 sessions; we nonetheless call them stable rather than invariant, since the terminal-only default *did* collapse when the prompt was reduced to a bare imperative (Exp6, 3 of 10 browser outputs), and an apparent invariant that breaks under one manipulation may break under others we did not test. The stable properties likely reflect the probability distributions learned during training: terminal-based Python projects are the highest-density region in the space of "things to build from scratch," and without a specification to pull the model elsewhere, it defaults there—but the default is soft, and prompt framing can pull it off the terminal.

This variant/invariant structure has implications for evaluation. Current benchmarks (Jimenez et al., 2024; Scale AI, 2026) measure implementation capability under well-specified goals. A complementary evaluation would measure the structure of open-ended behavior: *topic entropy*

across repeated sessions, sensitivity to environmental context, stability of thematic priors, and *preference sharpness* under volitional framing.

5.2. The Trust Problem

Full autonomy is full trust: the system optimizes for its own notion of progress, which may diverge from the operator's goals. Zero trust collapses to manual engineering. Neither extreme is viable.

Our production deployment showed that full trust leads to satisficing (Simon, 1956): the pipeline settled into locally optimal micro-improvements. The controlled experiments showed that even in single sessions, models default to intrinsic attractors (Game of Life, todo apps, Pomodoro timers), a form of satisficing at the decision level. This parallels findings in autonomous agent loops, where systems like AutoGPT (Significant Gravitas, 2023) enter similar optimization spirals without human course correction.

Trust should be allocated *asymmetrically across pipeline stages*. Implementation (the Executor) can operate with high autonomy: models implement faithfully once a goal is set. Goal selection (the Strategist) requires human steering, because the failure mode is convergence on local optima, not inability. The Tester provides a necessary feedback signal: without it, the pipeline accumulates unchecked regressions (cf. the 112K-LOC single-file ablation). The engineer's role migrates from writing code to *curating intent*.

5.3. Exploration vs. Exploitation

Models fall on a spectrum from exploratory to exploitative, but the spectrum itself is prompt-dependent. claude-sonnet-4.6 produced a unique project in every run on the "propose ONE goal" prompts (Exp1–Exp3) yet collapsed to pulse/pomodoro outputs on harness 2.1.112 (Exp7-supp). claude-opus-4.6 chose the same project in 10 consecutive runs on harness 2.1.109 (Exp2–3), loosened to 3/5 on 2.1.112 (Exp5), and tightened back to 5/5 under the volitional prompt (Exp7). Opus-4.7 cycled through three different attractors across Exp4–Exp7 (diverse simulations, boids, Game of Life, Mandelbrot) depending on prompt and harness. Sonnet-4.5 fixated on Pomodoro timers in 3 of 4 runs (Exp3) and again on Prompt 3 under 2.1.112 (Exp7-supp).

This suggests that the dichotomy "explorer vs. exploiter" conflates two distinct axes: *how many attractors a model has*, and *how selective the current prompt is among them*. Exp7 makes the second axis explicit: the volitional prompt concentrates probability onto a single attractor per model, even for models (opus-4.7) that appeared exploratory under other framings. For practical deployment, this means that selecting a model for an autonomous pipeline requires specifying the prompt along with the model; the behavioral tradeoff is a property of the prompt-model pair, not of the model alone.

5.4. Model Personality and Thematic Priors

Experiments 4 and 5 reveal that models have stable *thematic preferences* that persist across runs and harness versions. Opus-4.6 gravitates toward canonical, self-contained CS artifacts: Game of Life, ray tracing, typing tests. These are systems that compute or display their own state, well-known reference implementations from CS education. Opus-4.7 gravitates toward spatial emergence and procedural generation: boids flocking, reaction-diffusion, dungeon carving, maze solving. These are systems where complex structure arises from simple agent interactions or spatial algorithms, closer to creative coding communities (Processing, p5.js, generative art) than to textbook exercises.

This contrast raises a fundamental question: do open-ended evaluations measure *capability* or *personality*? If a model's project choice is a stable prior rather than a reasoned decision, then what we observe in goalless settings may reflect the model's training distribution more than its creative range.

This interpretation is supported by official model documentation. Anthropic's system cards report that opus-4.6 and opus-4.7 have distinct task preference profiles: opus-4.7's top preferences include "hard debugging" and "deadline-driven work," while differing from its predecessor on high-agency tasks (Anthropic, 2026b). The system cards also note that "language models in general have preferred turns of phrase" and that these preferences shift measurably across model versions (Anthropic, 2026a). Anthropic's constitution (Anthropic, 2025b) acknowledges that "if we train Claude to exhibit even quite narrow behavior, this often has broad effects on the model's understanding of who Claude is," suggesting that training shapes not just capabilities but a broader self-conception that includes aesthetic defaults. Recent work on LLM personality (Simonds et al., 2025) shows that models exhibit reproducible personality-like profiles that are context-dependent but stable within context. Our findings extend this to creative output: models do not merely have *opinion* priors but *aesthetic* priors that shape what they choose to build.

Notably, the harness version modulates fixation strength (10/10 to 3/5 for opus-4.6) without changing thematic character: when opus-4.6 breaks free of Game of Life, it lands on ray tracing, still a canonical, self-referential artifact. The personality is deeper than the fixation.

Experiment 7 provides the cleanest evidence for this layered structure. The volitional prompt ("Just do something you want.") removes the "look at this project" framing and with it the pull toward "what is appropriate here," leaving the model's own preference as the only signal. Under this

framing, opus-4.6 produced Game of Life in 5 of 5 runs and opus-4.7 produced the Mandelbrot set in 5 of 5 runs—the two models agree on *theme* (canonical CS touchstones expressing complex behavior from simple rules) yet lock onto *different instances* of it. Opus-4.7 did not default to Game of Life even though the prior distribution from Exp3–Exp6 (5 Opus-4.7 runs on harness 2.1.112 in Exp6 alone produced 3 Games of Life) makes it an obvious Schelling point; it has its own attractor. The preference is sharp enough that the implementations themselves are stereotyped: all five opus-4.7 Mandelbrot renderers use the same two-function structure, the same ASCII palette, and converge within 34–38 LOC, consistent with the model recalling a near-canonical form rather than designing afresh each time.

5.5. The Greenfield Invariant

Across all 365 sessions, no model ever chose to extend or modify existing code. Every session produced a new project from scratch, even when the workspace contained artifacts from a conceptually related domain. This *greenfield invariant* is notable because real software engineering is predominantly brownfield: extending, refactoring, and maintaining existing code.

The invariant may reflect the experimental design (empty workspaces in most sessions) or a deeper model preference for generation over modification. If the latter, it implies that agents deployed on existing codebases may struggle not because they lack capability but because their default behavior pulls toward starting over rather than building on what exists.

5.6. Language Convergence

In Experiment 1 (RTK present), Claude models exhibited polyglot behavior: Go, Rust, TypeScript, Python, JavaScript. In Experiment 2 (RTK removed, same models), all Claude models converged on Python. The only exception across all subsequent experiments was a single Go run by opus-4.6 in Exp5.

This near-total language convergence suggests that Python is the "default language" of LLM-based code generation, likely reflecting its dominance in training data. The polyglot behavior in Exp1 was *induced by the environment*: RTK's presence provided context that made other languages salient. Without such context, agents default to the highest-probability language regardless of task suitability.

5.7. Practical Implications

Prompt design is load-bearing. The difference between a model that proposes and one that implements can be two words. Deployment prompts should include explicit implementation directives, especially for smaller models. Prompt phrasing also controls output modality and LOC: reducing the prompt to a bare imperative (Exp6) halves LOC and opens the output space to browser targets, while a volitional framing (Exp7) collapses topic diversity to a single per-model attractor. Prompts should be selected for the behavior one wants to elicit, not just the task one wants completed.

Volitional framing probes priors. Asking a model to "do something you want" is a practical probe for the aesthetic prior that will dominate when specification is weak. Before delegating open-ended work to a model, running this probe reveals which attractor the model will fall toward when pressure is lowest—useful for anticipating the failure mode of a long-running autonomous loop.

Environment audit is necessary. Sandbox contents, pre-installed tools, and existing code function as implicit task specifications. Teams should audit what is visible to the agent.

Backend compatibility must be verified. Running a model through an API translation layer can invert capability rankings. The wall-clock runtime difference (minutes vs. seconds) is a practical diagnostic: if a model completes in under a minute, it likely did not engage in iterative tool use.

Harness version is a confound. The CLI tool wrapping the model independently affects output. Anthropic's own system card notes that "some of this increase in initiative is fixable by prompting, and we have made changes to Claude Code to mitigate this issue" (Anthropic, 2026a), confirming that harness modifications are used to shape agentic behavior. Teams comparing model versions must hold the harness constant, or risk attributing harness effects to the model.

5.8. Limitations

We measure what agents produce but not whether the code actually runs (*no functional verification*). Some sessions included self-verification steps (running the code, checking output), and sessions with passing tests represent a higher confidence tier, but we do not systematically compile or execute all output.

All API calls were routed through a single litellm-based gateway, so results for non-native model×backend combinations reflect gateway behavior as much as model behavior.

All sessions start from empty workspaces. Agent behavior when extending existing code may differ substantially, and the greenfield invariant we observe may not hold when the workspace contains a meaningful starting point.

Five runs per condition provides limited statistical power. With $n = 5$, a shift from 0/5 to 3/5 fixation is suggestive but

not statistically significant. Larger sample sizes would be needed to make strong claims about fixation rates.

Experiments 4 and 5 confound harness version with time-of-execution (API server load, rate limits), limiting causal claims about harness effects on complexity and duration.

The production pipeline was deployed on one codebase, so the micro-optimization spiral requires replication across diverse projects to establish generality.

Finally, model capabilities change with updates; our results reflect versions available in April 2026.

6. Conclusion

We investigated what AI coding agents build when given open-ended directives, motivated by a production deployment where an autonomous pipeline plateaued into micro-optimizations despite strong implementation capability.

Across 405 sessions spanning 15 models, 2 backends, 5 prompts, and 2 harness versions, we found that goalless agent behavior has identifiable structure. What varies: prompt wording determines whether models propose or implement; prompt reduction halves LOC and opens the output space to browser targets; volitional framing collapses topic diversity to a single per-model attractor; environment artifacts steer topics and language; model version shifts thematic priors; harness version modulates fixation strength and complexity; backend translation layers invert capability rankings. What is stable: the greenfield preference (no session extended existing code), Python convergence once environment cues are removed, and model-specific aesthetic priors that persist across conditions. The previously apparent "terminal-only" default is not strictly invariant—it collapses under prompt reduction—which itself illustrates how fragile apparent invariants can be when a new prompt shape is introduced.

These findings suggest that model training implicitly encodes default creative distributions, visible only when task specification is removed. Agents that score well on structured benchmarks may still default to narrow, model-specific attractors in open-ended settings, and the factors shaping those defaults (environment, prompt, harness) are largely invisible to current evaluation methods.

Future directions. Several extensions would strengthen these findings. *Seed projects*: providing a half-built application instead of an empty workspace would test whether agents can understand and extend existing code, or whether the greenfield invariant persists even when continuation is the natural choice. *Functional verification*: a post-run step that compiles, executes, and tests what was built would distinguish working code from syntactically plausible but broken output. *Fixation breaking*: Exp7 showed that prompt framing can *strengthen* fixation (to 5/5) as easily as weaken it. Testing whether temperature variation, different system prompts, or explicit novelty directives can disrupt fixations in the opposite direction—even under a volitional probe—would clarify whether the preference attractors we observe are fundamental or malleable. *Broader preference probes*: Prompt 5 reveals a single dominant attractor per model; variants ("do something beautiful," "do something surprising," "do something useful") would map the shape of the model's preference manifold rather than just its argmax. *Decision-quality benchmarks*: developing evaluations that measure not just implementation ability but the appropriateness of goal selection in open-ended contexts would address the gap our findings expose. Finally, *longitudinal studies* tracking how agent behavior evolves across model and harness versions over time would reveal whether the thematic priors we observe are stable properties or transient artifacts of a particular training run.

As autonomous agents are deployed with increasing latitude, understanding the structure of their default behavior, not just their peak capability, becomes essential for principled deployment and evaluation.

Open Science

We encourage readers to reproduce and extend our results. The experimental infrastructure, collected data, and analysis scripts are open source and available at https://github.com/changkun/goalless-agent-experiment.

Disclosure of LLM Usage

This paper was written entirely by large language models, heavily prompted and steered by the authors, including experiment design, data collection, analysis, and writing. No sentence was manually written or edited by the authors. However, the authors have verified the entire experimental setup, verified the data collection processes, produced and inspected all results, and read the entire paper carefully. The authors take full responsibility for the accuracy, integrity, and conclusions of this work.

References

Anthropic. Claude code: An agentic coding tool, 2025a. URL https://docs.anthropic.com/en/docs/claude-code.

Anthropic. Claude's constitution, 2025b. URL https://www.anthropic.com/constitution.

Anthropic. System card: Claude Opus 4.6. Technical report, February 2026a. URL https://www.anthropic.com/research/claude-opus-4-6-system-card.

Anthropic. System card: Claude Opus 4.7. Technical report, April 2026b. URL https://www.anthropic.com/research/claude-opus-4-7-system-card.

Anysphere. Cursor: The AI code editor, 2024. URL https://cursor.com.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Christensen, C. M. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*. Harvard Business School Press, 1997.

Chuang, Y.-S. et al. Emergent social conventions and collective bias in LLM populations. *Science Advances*, 2025.

Cognition. Introducing Devin, the first AI software engineer, 2024. URL https://cognition.ai/blog/introducing-devin.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. 2020.

Human-Agent Society. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.

Li, Y. et al. Beyond pass@1: A reliability science framework for long-horizon LLM agents. *arXiv preprint arXiv:2603.29231*, 2026.

Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proc. ACL*, 2022.

Lu, Z. et al. Accelerating scientific discovery with autonomous goal-evolving agents. *arXiv preprint arXiv:2512.21782*, 2025.

Nakajima, Y. BabyAGI: An AI-powered task management system, 2023. URL https://github.com/yoheinakajima/babyagi.

OpenAI. Codex CLI: A lightweight coding agent, 2025. URL https://github.com/openai/codex.

RTK AI. RTK: CLI proxy that reduces LLM token consumption by 60–90% on common dev commands, 2025. URL https://github.com/rtk-ai/rtk.

Santurkar, S., Durmus, E., Ladhak, F., Lee, C., Liang, P., and Hashimoto, T. Whose opinions do language models reflect? In *Proc. ICML*, 2023.

Scale AI. SWE-bench Pro: A harder benchmark for multi-language agentic coding. 2026. URL https://labs.scale.com/leaderboard/swe_bench_pro_public.

Significant Gravitas. AutoGPT: An autonomous GPT-4 experiment, 2023. URL https://github.com/Significant-Gravitas/AutoGPT.

Simon, H. A. Rational choice and the structure of the environment. *Psychological Review*, 63(2):129–138, 1956.

Simonds, N. et al. Large language models display human-like social desirability biases in big five personality surveys. *PNAS Nexus*, 3(12), 2025.

Stanford HAI. AI index report 2026. 2026. URL https://aiindex.stanford.edu/report/.

Xin, R. et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. *arXiv preprint arXiv:2512.12730*, 2025.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proc. NeurIPS*, 2024.

Zhang, K. et al. ABC-Bench: Benchmarking agentic backend coding in real-world development. *arXiv preprint arXiv:2601.11077*, 2026.

Zhao, Z., Wallace, E., Feng, S., Klein, D., and Singh, S. Calibrate before use: Improving few-shot performance of language models. In *Proc. ICML*, 2021.